

Developer Report

Test Environment

All findings were reproduced using Chrome DevTools on the AP News homepage.

Tools used

- Lighthouse
 - Performance
 - Coverage
 - Network
 - Layers
 - JavaScript-disabled testing
-

Finding 1 – Excessive JavaScript Execution

Mechanism

JavaScript execution dominates the main thread during the initial page load. Coverage analysis indicates that a significant portion of downloaded JavaScript is not required for the initial render, increasing main-thread work before the page becomes fully interactive.

Evidence / Reproduction

Open DevTools → Performance.

Record a page reload.

Observed:

- Scripting **≈ 3.1 s**
- Rendering **≈ 398 ms**
- Painting **≈ 215 ms**

Coverage analysis shows a substantial amount of unused JavaScript during the initial page load.

Recommended Fix

- Improve bundle splitting.
- Remove unused JavaScript.
- Defer non-essential scripts.
- Delay optional functionality until after the primary content has rendered.

Verification

Repeat Performance and Coverage analysis.

Expected improvements:

- Reduced scripting time.
- Reduced unused JavaScript.
- Improved responsiveness.

Estimated Effort

Medium

Scope

Structural

Finding 2 – Third-Party Services

Mechanism

Advertising, analytics, and tracking services contribute additional JavaScript execution during page initialization, increasing CPU utilisation and delaying interactivity.

Evidence / Reproduction

Record a Performance profile.

The flame chart shows execution associated with multiple third-party advertising and analytics services.

Coverage also identifies unused JavaScript from several third-party resources.

Recommended Fix

- Audit all third-party services.
- Remove unused integrations.
- Delay non-essential advertising and analytics until after the initial page render where possible.

Verification

Record another Performance profile.

Confirm:

- Reduced third-party scripting.
- Lower Total Blocking Time.
- Reduced CPU usage.

Estimated Effort

Low–Medium

Scope

Mostly Local

Finding 3 – Unused CSS and JavaScript

Mechanism

Coverage analysis identifies CSS and JavaScript resources that are downloaded but not required during the initial rendering process.

Evidence / Reproduction

Open DevTools → Coverage.

Reload the homepage.

Observed:

- Unused CSS.
- Unused JavaScript during initial rendering.

Recommended Fix

- Remove unused CSS rules.
- Defer non-critical styles.
- Continue reducing unused JavaScript bundles.

Verification

Repeat Coverage analysis.

Expected improvements:

- Lower unused resource percentage.
- Smaller transferred assets.
- Faster initial rendering.

Estimated Effort

Medium

Scope

Mostly Local

Finding 4 – Content Delivery Strategy

Mechanism

The homepage uses **Server-Side Rendering (SSR)** with client-side hydration. Response headers also indicate that Cloudflare caches the generated HTML for short periods before serving it to users.

Evidence / Reproduction

Disable JavaScript.

Reload the homepage.

Observed:

- News content remains visible.
- HTML source already contains rendered content.
- Response headers include Cloudflare caching.

Recommended Action

No change recommended.

The current rendering strategy is appropriate for a news website. Optimisation efforts should focus on reducing hydration costs and JavaScript execution rather than replacing SSR.

Verification

Not applicable.

Estimated Effort

None

Scope

Not Applicable